Date 28/07/2026

Expt. No. 1 Page No. 1

Aim:- To sort a given set of elements using the Quick Sort method, determine the time required to sort the elements for different values of n, & plot a graph of time taken versus n.

Theory:-

Quick sort is a comparison-based sorting algorithm based on the Devide & conquer technique It works be selecting one elements as a pivot & rearranging the array such that

- element smaller then pivot placed on left
- element greater then pivot placed on right.

The pivot is then placed at its correct position.

Pivot The element selected for partitioning the array.

Ex: 5, 2, 8, 1, 7

5 is pivot

2 1 [5] 8 7

Partition Partition rearranges the elements around the pivot

(element<pivot) | pivot | (element>pivot)

Teacher's Signature _____

Algorithm

```
QuickSort ( array , low, high)
    if low < high
        pivotIndex = partition (array, low, high)
        QuickSort (array, low, pivotIndex -1)  //left
        QuickSort (array, pivotIndex +1, high) //right

partition(array , low, high )
    pivot = array[high]
    i = low -1

    for j from low to high -1
        if array[j] < pivot
        i++
        swap array[ i ] with array [j]

    Swap array[i+1] with array [high]
    return i+1
```

Java Code

```
public class Main {
  private static int partition(int [] arr, low, int high)
{
  int pivot = arr[high];
  int i = (low -1);

  for(int j = low ; j < high ; j++){
      if ( arr[j] < pivot ) {
          i++;
```

Teacher's Signature

Output

Sorted array: 1, 2, 3, 5, 8

Expt. No. _____ Date _____

Page No. 3.

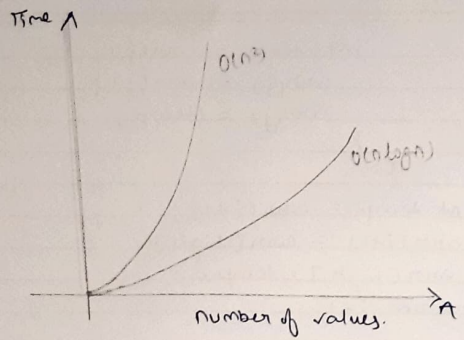
```
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i+1];
    arr[i+1] = arr[high];
    arr[high] = temp;
    return i+1;
}

public static void quickSort(int arr[], int low, int high)
{
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
        quickSort(arr, low, pivotIndex-1);
        quickSort(arr, pivotIndex+1, high);
    }
}

public static void main(String[] args) {
    int[] arr = {5,2,8,1,3};
    int n = arr.length;
    quickSort(arr, 0, n-1);

    System.out.print("Sorted Array ");
    for (int val: arr){
        System.out.print(val + " ");
    }
}
}
```

Teacher's Signature _____



Date

Expt. No.

Page No. 4.

Time Complexity

Best Case = O(n log n)
Average Case = O(n log n)
Worst Case = O(n²)

Space Complexity

Average Case = O(log n)
Worst Case = O(n)

Result Thus, Quick sort was successfully implemented & the time required to sort different numbers of elements was measured. The experimented result show that Quick sort generally perform in O(n log n) times for average input

Viva Question

Q1 What is Quick sort?
Ans Quick sort is a Divide & Conquer based sorting algorithm the partitions the array around a pivot.

Q2 What is a pivot?
Ans Pivot is the element around which the array is partitioned?

Q3 What are the average & worst case Time Complexity of Quick Sort?
Ans Worst Case = O(n²)
Average Case = O(n log n)

Teacher's Signature

Q4 Why does worst case occur in quick sort?

Ans When the selected pivot produces highly unbalanced partitions, such as:

0 element | pivot | (n-1) elements.

repeatedly.

Q5 Is Quick sort stable or in-place?

Generally Quick sort is not stable. The common implementation is considered as in-place sorting algorithm, apart from recursion-stack space.